Project Report: File Upload System

EC2 Web Application to S3 Storage via AWS IAM Role

Course / Subject: Cloud Computing / AWS	**Name:** Amarnath K
AWS Region: Europe (Ireland) us-west-1	**Date:** October 2026

1. Introduction

Amazon Web Services (AWS) provides robust cloud infrastructure for building secure, scalable applications. This report documents the implementation of a full-stack file upload application hosted on an Amazon EC2 instance that uploads user-selected files directly to an Amazon S3 bucket. Security is maintained by attaching an IAM Role directly to the EC2 instance, eliminating the need for hardcoded AWS credentials in the backend application code.

2. Step-by-Step Procedure

Step 1: Creating the Target S3 Bucket

1. Navigate to the Amazon S3 Console and click **Create bucket**.
2. Set the Bucket Name: **project-s3-iam**.
3. Uncheck **Block all public access** (for demonstration and lab testing purposes).
4. Click **Create bucket** to initialize object storage.

Step 2: Creating an IAM Role for EC2

1. Open the IAM Console, navigate to **Roles**, and click **Create role**.
2. Select **AWS service** as the trusted entity type and choose **EC2** as the use case.
3. Attach the permission policy **AmazonS3FullAccess**.
4. Name the role **pro-s3-iam** and complete the role creation.

Step 3: Launching and Configuring the EC2 Instance

1. Navigate to the EC2 Console and launch a new instance running **Amazon Linux 2023** on a **t2.micro** instance type.
2. Under **IAM instance profile**, attach the newly created **pro-s3-iam**.
3. Configure the Security Group inbound rules to allow:
   - SSH: Port 22 (Your IP)
   - HTTP: Port 80 (0.0.0.0/0)
4. Launch the instance and connect via SSH.

Step 4: Installing Required Software Dependencies

1. Connect to the EC2 instance and update installed packages:

```
sudo yum update -y
```

2. Install Node.js and npm:

```
sudo yum install -y nodejs npm
```

3. Install the required Node.js packages (express, multer, and aws-sdk):

```
npm install express multer aws-sdk
```

Step 5: Developing the Node.js Backend Application

1. Create an application directory and navigate into it:

```
mkdir upload-app && cd upload-app
```

2. Create **app.js** and implement the Express server code:

```
const express = require("express");
const multer = require("multer");
const AWS = require("aws-sdk");
const fs = require("fs");

const app = express();
const upload = multer({ dest: "uploads/" });
const s3 = new AWS.S3();

app.post("/upload", upload.single("file"), (req, res) => {
  const file = req.file;
  const fileContent = fs.readFileSync(file.path);

  const params = {
    Bucket: "file-upload-bucket-<yourname>",
    Key: file.originalname,
    Body: fileContent,
  };

  s3.upload(params, (err, data) => {
    fs.unlinkSync(file.path); // remove temporary local file
    if (err) {
      res.status(500).send("Error uploading file");
```

```
    } else {
      res.send("File uploaded successfully to S3: " + data.Location);
    }
  });
});

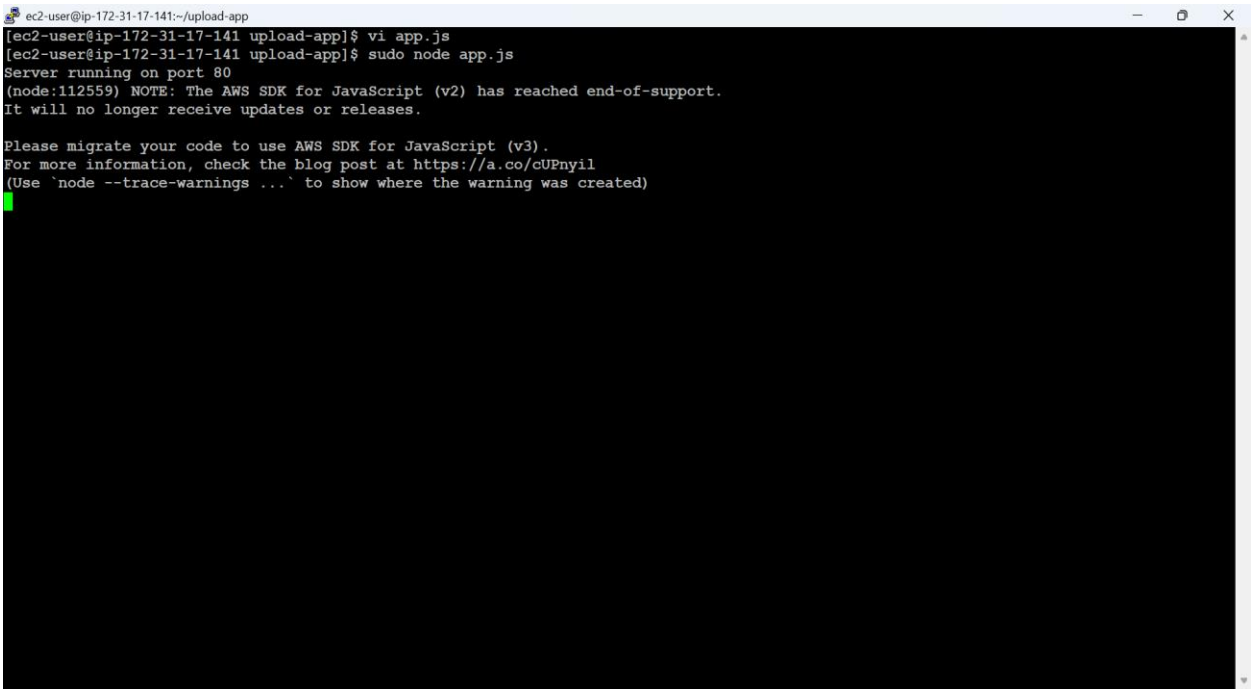
app.listen(80, () => console.log("Server running on port 80"));
```

Step 6: Starting the Backend Server

1. Start the Node.js server with administrative privileges to bind to Port 80:

```
sudo node app.js
```

2. Verify that the terminal out



puts **Server running on port 80**.

Step 7: Creating the Frontend User Interface

1. Create an HTML file named **index.html** with a file selection form targeting the EC2 Public IP address:

```
<!DOCTYPE html>
<html>
<head>
  <title>Upload to S3</title>
</head>
```

```
<body>
  <h2>Upload File to S3</h2>
  <form action="http://<EC2_PUBLIC_IP>/upload" method="POST" enctype="multipart/form-
data">
    <input type="file" name="file" required><br><br>
    <button type="submit">Upload</button>
  </form>
</body>
</html>
```

Step 8: Uploading a File and Testing the Application

1. Open **index.html** in a Web Browser.
2. Click **Choose File**, select a local image or document, and click **Upload**.
3. Verify that the page returns the successful S3 upload response containing the object URL:
   File uploaded successfully to S3: https://file-upload-bucket.../photo.png
4. Check the Amazon S3 console to confirm that the file is present in the target bucket.

Step 9: Verifying IAM Role Privileges via EC2 CLI

1. Execute the AWS S3 CLI listing command inside the EC2 instance:

```
aws s3 ls s3://upload image photo-Amarsai
```

2. Confirm that uploaded files are listed without entering AWS secret keys, validating that the assigned IAM Role is active.

3. Verification & Summary

Target S3 Bucket	project-s3-iam
Attached IAM Role	EC2-Upload-To-S3-Role (AmazonS3FullAccess)
Runtime Environment	Node.js (Express + Multer + AWS SDK) on Amazon Linux 2023
Network & Security	HTTP Traffic allowed over Port 80
Execution Status	Server active on Port 80, processed file upload request, saved objects to S3, permissions verified via CLI

4. Conclusion

The end-to-end file upload application was successfully built and deployed on an AWS EC2 instance. By attaching an IAM Role directly to the EC2 compute instance, the application safely authenticated with Amazon S3 using temporary metadata credentials, eliminating security risks associated with hardcoded IAM credentials.